Investigating Dynamic vs Static Thread Pools for Performance in Multithreaded Applications

- Alfredo Ortiz, Harshil Patel, Kaichi Xie, Anugya Sharma

UCDAVIS

Background: Static Vs Dynamic Thread pool

Each thread pool has there own strength and setbacks.

Static thread pools:

- Pre-created threads
- Waste memory during low load and idle periods.

Dynamic thread pools:

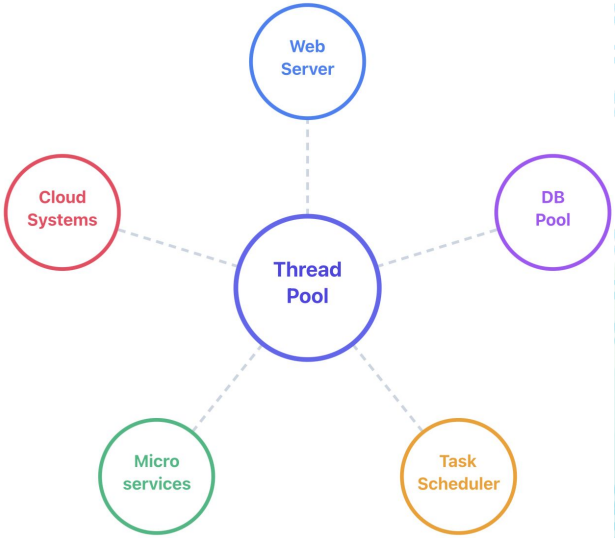
- Threads created on demand
- Avoids memory waste by creating threads only when needed.
- Incurs overhead for thread creation and destruction.
- Performance degradation under high load.

It is important to pick the right thread pool for the job

Why it matters?

Some thread pools will perform better than another in different scenarios:

- Variable traffic scenarios: A network that gets burst traffic needs a pool strategy that handles both idle periods and sudden spikes efficiently. (10.1109/ICCCS55155.2022.9846591)
- Affects design of real world systems like database connection pools, task scheduling in distributed systems etc. which all use thread pools to manage concurrency.
- Not many studies on thread pool for software program used on a local computer.



Which thread pool is better for software application?

- Our focus is solely on software performance instead of network performance.
- Our test focused on time, and memory.
- We wanted to find which thread pool is best for software application since no one thread pool method is best for all scenarios.
- If we want to maximize the performance, we have to pick the right thread pool for the scenario.

Hardware Specification

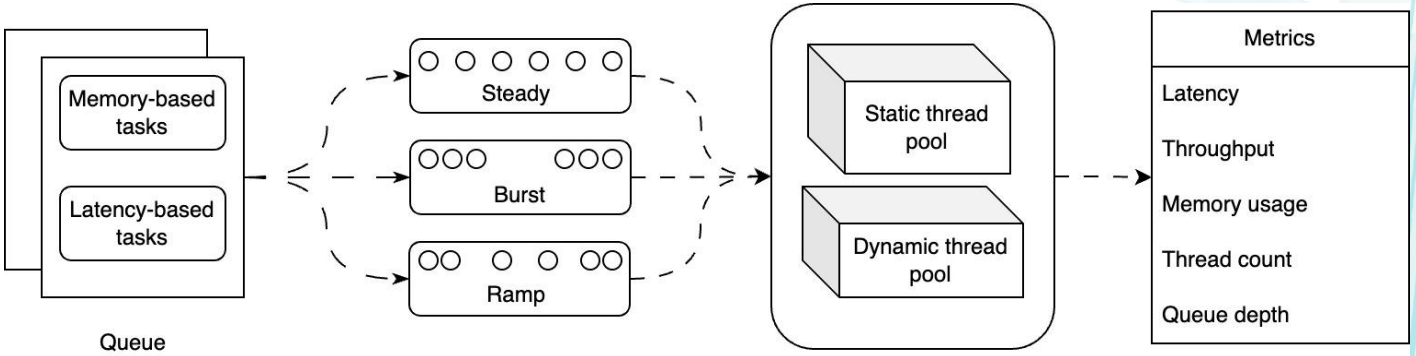
Cloud lab Specs:

- CPU: two Intel Xeon Silver 4114 CPU
  - 10 core (each cpu)
  - 2.20 GHz
- RAM: 192GB DDR4-2666 memory
- OS: UBUNTU 24.04

Approach overview

Components:

1. FIFO Queue: three arrival patterns
2. Tasks: Matrix multiplication (memory-base), Sleep (latency-based)
3. Thread pools: static and dynamic



Thread Pool Implementation

1. Thread Safe Task Queue
2. Static thread pool
   a. Fixed number of threads created at initialization, each takes tasks from the shared task queue.
   b. Threads block when the queue is empty.
3. Dynamic thread pool
   a. Has minimum threads spawned (2)
   b. Spawns new threads every X= 10 tasks in the Task Queue
   c. Destroys the thread if it is Idle for more than Y= 200ms time.
   d. The thread creation caps out at the OS limit
   e. We decided to not cap the thread count as we want to compare the thread pools and at a cap the dynamic pool is essentially a static thread pool.

Experiment setup

Arrival patterns:

1. **Steady**: a constant arrival rate of 1000 tasks/sec.
2. **Burst**: a burst of 2000 tasks/sec followed by a 2-sec idle period.
3. **Ramp**: alternating phases on 1500 tasks/sec and 500 tasks/sec every 20 sec.

Benchmark tasks:

1. **Memory-based**: N x N matrix multiplication (128, 256, 512)
2. **Latency-based**: Sleep(T) blocks (100μs, 1 ms, 10ms)

Total configurations:

We conducted 3 runs per configuration (arrival patterns (3) x benchmark tasks (2) x hyperparameter (3) x thread pool types (2) = **36** total configuration for each run) to account for confounding variabilities in

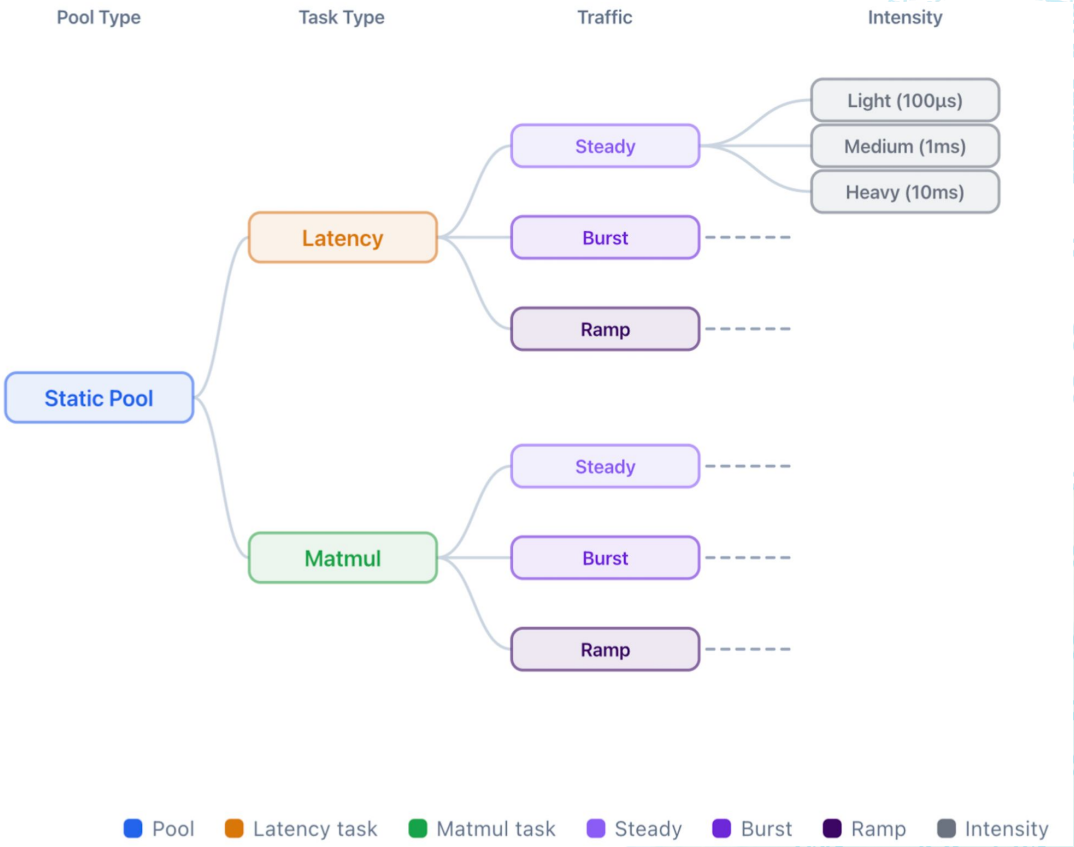
Evaluation Metrics

1. Memory usage (in KB): min, max, mean.
2. Latency (in ms): min, max, mean, std_dev, p50, p90, p99
3. Throughput (in ms)
4. Queue depth: min, max, mean, p95, p99
5. Thread counts: min, max, mean

Results

- Collected metrics data across 108 runs. (3 combination of pool, task, traffic and intensity)
- Evaluation divided between memory and time based.
- 0.2 second sampling. (Mean or p99)

Metric	Measure (Mean of 3 runs)
Throughput	**mean**, max, min (tasks/s)
Latency	mean, max, p95, **p99** (ms)
Memory	**mean**, max, min (KB)
Thread Count	**mean**, max, min
Queue Depth	mean, max, p95, **p99** (ms)



Memory Based Task

- **Matrix multiplication**
- **Measure selection for comparison: Either mean or p99**
- **Light load (128 x 128):**
  - Similar performance, Both pool match the target throughput (~999 tasks/s) across all traffic patterns.
  - Sub- 7ms p99 latency and queue depth stays at 0-1
  - Memory usage is comparable: 9-21 MB
  - Exception at Burst Traffic : 2 times more memory usage by dynamic, high no. of threads spawned but tasks finish quick.

Intensity	Traffic Pattern	Static Throughput (tasks/s)	Dynamic Throughput (tasks/s)	Static Latency p99 (ms)	Dynamic Latency p99 (ms)	Static Memory Mean (KB)	Dynamic Memory Mean (KB)	Static Threads (mean)	Dynamic Threads (mean)	Static Queue p99	Dynamic Queue p99
Low 128x128	burst	998.99	998.88	5.74	6.62	7972.86	16946.68	12.0	17.93	1.0	1.0
	ramp	998.92	998.91	6.56	6.59	8154.5	8788.77	12.0	13.98	0.67	1.0
	steady	998.86	998.85	6.6	6.55	8042.59	7452.86	12.0	9.99	1.0	1.0

Memory Based Task

Observations for Medium Load (256 x 256) and **Heavy Load (512 x 512)**

- **Good Latency:** Dynamic always wins in latency.
- **Okay Throughput:** For medium(N=256) tasks, Static pool sustains ~628 tasks/s whereas Dynamic pool reaches only ~185-253 tasks/s. But dynamic pool catches up at high intensity task.
- **Worst Memory:** Memory diverges sharply. Static pool stays under 75 MB but dynamic balloons to 3.7 GB (N=256) and upto 27 GB (N=512).

Intensity	Traffic Pattern	Static Throughput (tasks/s)	Dynamic Throughput (tasks/s)	Static Latency p99 (ms)	Dynamic Latency p99 (ms)	Static Memory Mean (KB)	Dynamic Memory Mean (KB)	Static Threads (mean)	Dynamic Threads (mean)	Static Queue p99	Dynamic Queue p99
Medium 256x256	burst	628.47	252.03	71463.92	7267.56	21567.33	559864.46	12.0	23420.44	44997.0	2040.67
	ramp	628.47	253.33	72535.67	44085.01	21642.87	2091171.59	12.0	137086.41	45630.0	11911.67
	steady	628.01	185.08	70277.6	38719.17	21455.35	2156346.22	12.0	150466.31	44135.0	12715.67
High 512x512	burst	65.8	62.18	1687830.86	48941.92	72112.85	4820670.88	12.0	7125.43	111072.0	1596.33
	ramp	65.87	60.35	1685673.8	151306.84	72127.96	7219700.47	12.0	43557.87	111081.0	5767.67
	steady	65.85	55.07	1685253.26	841820.59	72100.61	7395139.24	12.0	133053.42	111002.0	48718.67

Memory Based Task

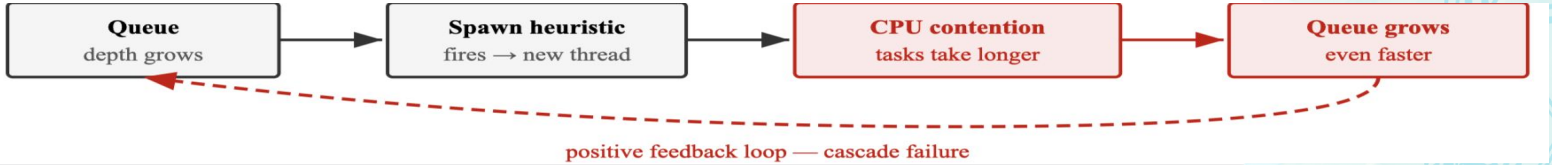
Insights: Medium Load (256 x 256) and **Heavy Load (512 x 512)**

How does dynamic perform better at heavy load tasks?

- Task duration vs Thread creation overhead : When tasks take much longer to complete, thread creation cost becomes a tiny fraction of total task time. Dynamic pool's thread creation overheads are finally justified.

Why 100X memory usage in Dynamic?

- Static pool persistently works the 10 thread, memory under 72 MB.
- Dynamic pool's memory usage is in GBs, stuck in a vicious cycle of (threads spawned - cpu maxed out - more threads spawned).
- CPU contention, cache thrashing, and memory pressure that degrades throughput for dynamic threadpool.



Time-Based Tasks (Sleep/Latency)

Light (100μs) and Medium (1ms)

- Both pools are identical: ~999 tasks/s, p99 under 1.1ms, queue depth ≈ 0
- No benefit or cost to dynamic scaling at these durations

Intensity	Traffic Pattern	Static Throughput (tasks/s)	Dynamic Throughput (tasks/s)	Static Latency p99 (ms)	Dynamic Latency p99 (ms)	Static Memory Mean (KB)	Dynamic Memory Mean (KB)	Static Threads (mean)	Dynamic Threads (mean)	Static Queue p99	Dynamic Queue p99
Low	burst	998.9	998.92	0.2	0.2	4822.26	4672.47	12.0	4.04	1.0	1.0
	ramp	998.91	998.93	0.2	0.2	4847.68	4608.9	12.0	4.94	1.0	1.0
	steady	998.9	998.92	0.19	0.19	4812.1	4623.11	12.0	4.45	0.67	1.0
Medium	burst	998.87	998.9	1.1	1.1	4809.04	4832.77	12.0	4.71	1.0	1.0
	ramp	998.91	998.91	1.11	1.1	4829.68	4659.46	12.0	5.23	1.0	1.0
	steady	998.9	998.92	1.09	1.1	4816.06	4598.42	12.0	4.91	1.0	1.0

Time-Based Tasks (Sleep/Latency)

Heavy (10 ms)

- Static pool saturates: 10 threads × 10ms = exactly 1000 tasks/s ceiling with zero headroom and so the Queue depth spikes
- Dynamic pool handles heavy tasks perfectly
- Memory usage stays similar for both pools (~5–6 MB) since sleep tasks carry no per-task allocation

Intensity	Traffic Pattern	Static Throughput (tasks/s)	Dynamic Throughput (tasks/s)	Static Latency p99 (ms)	Dynamic Latency p99 (ms)	Static Memory Mean (KB)	Dynamic Memory Mean (KB)	Static Threads (mean)	Dynamic Threads (mean)	Static Queue p99	Dynamic Queue p99
High	burst	992.32	998.91	2591.75	10.1	4996.22	5118.15	12.0	15.54	2574.33	1.0
	ramp	992.32	998.91	5617.0	10.11	5168.22	4859.56	12.0	19.21	5543.67	1.0
	steady	992.32	998.92	865.42	10.1	4870.15	4871.47	12.0	13.44	849.33	0.67

Conclusions

- Dynamic pool wins for I/O-bound / latency-sensitive workloads : thread scaling prevents queue buildup when tasks can block; the heavier the task and the tighter the arrival rate, the larger the advantage
- Static pool wins for CPU-bound workloads: a fixed thread count matching available cores avoids context-switching overhead, cache thrashing, and memory bloat from excess threads.
- Thread over-scaling is harmful under compute pressure: spawning threads beyond core count produces diminishing returns then actively hurts throughput; memory from concurrent large allocations can exceed system RAM and trigger swapping
- Traffic pattern matters only when the pool is near saturation : Burst and Ramp expose queuing instability in the static pool under heavy latency tasks, but have negligible effect when throughput is the bottleneck (e.g. large matmul)

Thank You!